

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

Reverse Engineering Task

Code of Conduct:

I Sayed Fazal (192311291) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

Answer:

1. Identified Classes

Class	Main Attributes	Main Methods
Student	studentId, name, email, password	register(), login(), enrollCourse(), submitAssignment()
Instructor	instructorId, name, email	createCourse(), uploadContent(), evaluateAssignment()
Course	courseId, title, description, duration	createCourse(), updateCourse(), addContent()
CourseContent	contentId, title, type, filePath	uploadContent(), viewContent()
Enrollment	enrollmentId, enrollmentDate, status	enroll(), cancelEnrollment()
Assignment	assignmentId, title, deadline, marks	createAssignment(), submit(), evaluate()
Submission	submissionId, submissionDate, filePath, marks	submit(), evaluate()
Result	resultId, marks, grade, status	calculateGrade(), publishResult()

2. Main Relationships

Student 1 — * Enrollment

Course 1 — * Enrollment

Instructor 1 — * Course

Course 1 — * CourseContent

Course 1 — * Assignment

Student 1 — * Submission

Assignment 1 — * Submission

Student 1 — * Result

Course 1 — * Result

3. Responsibilities of Layers

Presentation Layer:

Handles screens, forms, dashboards, and user input. It does not contain business rules.

Business Layer:

Contains application logic such as enrollment validation, assignment evaluation, and result generation.

Domain Layer:

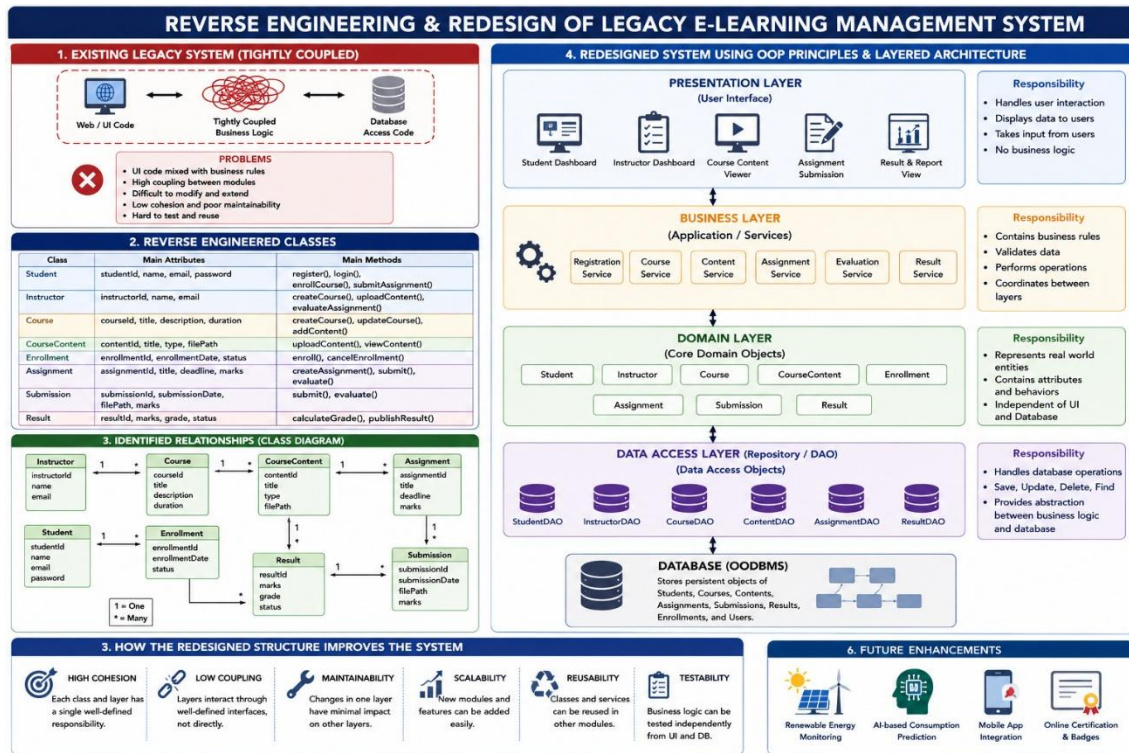
Contains the core objects and their data/behavior, such as Student, Course, Assignment, and Result.

Data Access Layer:

Handles database operations such as saving, updating, searching, and retrieving objects.

4. Improvements After Redesign

- **High Cohesion:** Each class performs a focused responsibility. For example, Assignment manages assignment-related behavior rather than student registration.
- **Low Coupling:** The UI does not directly depend on business rules or database operations.
- **Maintainability:** Changes to one module have minimal effect on other modules.
- **Scalability:** New features such as online examinations, certificates, discussion forums, or video courses can be added more easily.
- **Reusability:** Common classes and services can be reused across different modules.
- **Testability:** Business logic can be tested independently from the user interface and database.



2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

Answer:

1. Existing Design Limitations

- **High coupling:** The main system is directly dependent on every support channel.
- **Poor extensibility:** Adding a new channel requires modifying existing code.
- **Low maintainability:** Large conditional statements become difficult to understand and maintain.
- **Poor reusability:** Channel-specific logic cannot be easily reused.
- **Violation of Open/Closed Principle:** Existing classes must be modified whenever a new support channel is introduced.
- **Difficult testing:** All channel conditions are mixed together, making individual components difficult to test.

2. Reverse-Engineered Classes

Class	Responsibilities	Main Methods
SupportRequest	Stores customer support information	createRequest(), getDetails()
SupportManager	Processes and routes requests	handleRequest(), assignAgent()
EmailChannel	Handles email requests	receiveRequest(), sendResponse()
ChatbotChannel	Handles chatbot requests	receiveRequest(), sendResponse()
TelephoneChannel	Handles telephone requests	receiveRequest(), sendResponse()
SocialMediaChannel	Handles social-media requests	receiveRequest(), sendResponse()
SupportAgent	Resolves customer issues	handleRequest(), respond()

3. Suitable Behavioral Design Patterns

a) Strategy Pattern

The **Strategy Pattern** can represent different support-channel handling strategies.

SupportChannelStrategy

/ | | \

/ | | \

Email Chatbot Telephone SocialMedia

Each channel implements the same interface:

receiveRequest()

sendResponse()

The Support Manager can select the required strategy without using a large conditional statement.

Benefit: A new channel such as WhatsApp can be added as a new strategy without modifying the existing support manager.

b) Chain of Responsibility Pattern

The **Chain of Responsibility** can be used to process and route support requests through different support handlers.

Customer Request



Email Handler



Chatbot Handler



Telephone Handler



Social Media Handler



Support Agent

c) Observer Pattern

Support Request



notify()



+-----+-----+



Support Agent Support Manager

4. Refactored Structure

Customer



Support Request



Support Manager



Support Channel Strategy

└─ Email

└─ Chatbot

└─ Telephone

└─ Social Media



Support Handler Chain



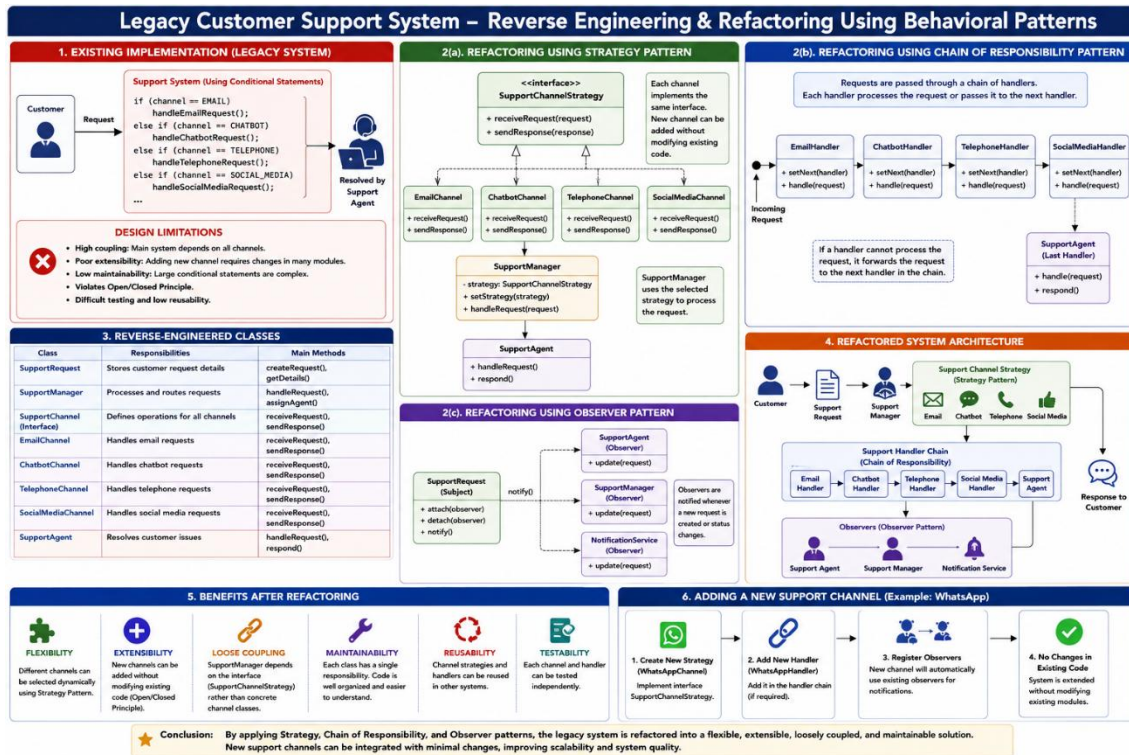
Support Agent



Response

5. Improvements After Refactoring

Requirement	Improvement
Flexibility	Different support channels can be selected dynamically.
Extensibility	New channels can be added without modifying existing channel logic.
Loose Coupling	Support Manager depends on interfaces rather than concrete channels.
Maintainability	Each channel and handler has a clear responsibility.
Reusability	Channel strategies and handlers can be reused.
Testability	Each channel can be tested independently.



3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes**, **service classes**, and **data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

Answer:

1. Reverse-Engineered Classes

Domain Classes:

- Customer** – customerId, name, email; getDetails()
- Account** – accountId, accountType, balance, status; deposit(), withdraw(), getBalance()
- Transaction** – transactionId, amount, date, type, status; createTransaction(), updateStatus()
- FundTransfer** – transferId, sourceAccount, targetAccount, amount; transfer()
- Loan** – loanId, amount, interestRate, status; applyLoan(), calculateInterest()

2. Service Classes

Business operations are separated into services:

- **AccountService** – account creation and account management.
- **TransactionService** – transaction validation and processing.
- **TransferService** – fund transfer operations.
- **LoanService** – loan processing and interest calculation.
- **ReportService** – transaction reporting.

3. Data Access Components

Database operations are moved into separate repositories/DAOs:

- CustomerRepository
- AccountRepository
- TransactionRepository
- LoanRepository

They provide operations such as:

save(), findById(), update(), delete(), findAll()

4. Redesigned Layered Architecture

Presentation Layer



Service / Business Layer



Domain Layer



Repository / DAO Layer



Database

- **Presentation Layer:** Handles user requests and displays results.
- **Service Layer:** Contains banking business rules and validations.
- **Domain Layer:** Represents Customer, Account, Transaction, Loan, etc.
- **Data Access Layer:** Handles all database operations.

5. Suitable Design Patterns

Repository/DAO Pattern:

Separates database operations from business logic.

Factory Pattern:

Can be used to create different account or transaction types.

Strategy Pattern:

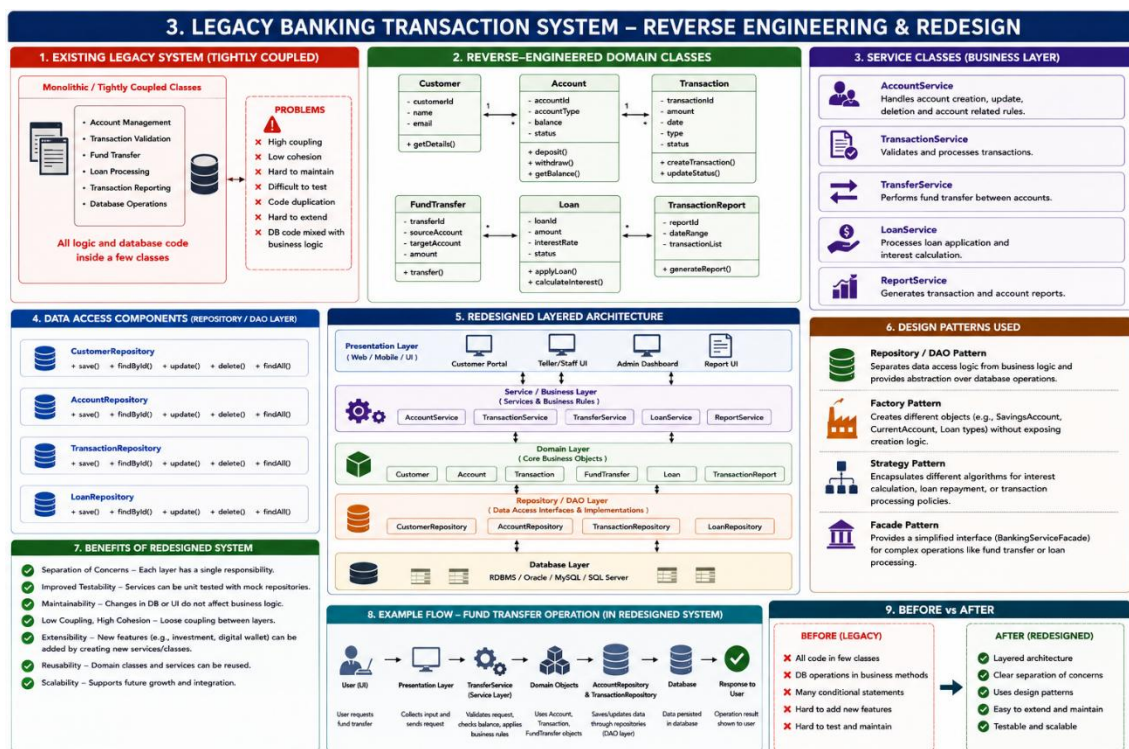
Can be used for different loan-interest or transaction-processing policies.

Facade Pattern:

A `BankingServiceFacade` can provide a simple interface to complex operations such as fund transfer and loan processing.

6. Improvements

- **Separation of Concerns:** Each layer has a specific responsibility.
- **Testability:** Service classes can be tested independently using mock repositories.
- **Maintainability:** Database changes do not require changes to business logic.
- **Low Coupling:** Services depend on repository interfaces rather than database implementations.
- **High Cohesion:** Each class focuses on a specific banking function.
- **Scalability:** New services such as investment management or digital wallets can be added without redesigning the entire system.



4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

Answer:

1. Problems in the Existing Architecture

- **High Coupling:** The user interface is directly dependent on the OODBMS.
- **Code Duplication:** Queries for rooms, guests, bookings, and payments are repeated.
- **Poor Maintainability:** A database change requires modifications in multiple modules.
- **Low Reusability:** Database operations cannot be easily reused by other services.
- **Poor Separation of Concerns:** UI, business logic, and persistence logic are mixed.
- **Difficult Testing:** UI modules cannot be easily tested without database access.

2. Reverse-Engineered Classes

Domain Classes:

- Guest – guestId, name, email, phone
- Room – roomId, roomType, price, status
- Booking – bookingId, checkIn, checkOut, status
- Payment – paymentId, amount, paymentDate, status

Service Classes:

- ReservationService
- RoomService
- GuestService
- PaymentService

3. Data Access Components

GuestRepository

RoomRepository

BookingRepository

PaymentRepository

Example operations:

RoomRepository

+ save()

+ findById()

+ findAvailableRooms()

+ update()

+ delete()

BookingRepository

+ save()

+ findById()

+ findByGuest()

+ update()

+ delete()

4. Redesigned Architecture

Presentation Layer

(Web / Mobile / Reservation UI)



Service Layer



Domain Layer



Repository / DAO Layer



OODBMS

- **Presentation Layer:** Handles user interaction and reservation screens.
- **Service Layer:** Handles reservation, room availability, guest, and payment logic.

- **Domain Layer:** Contains `Guest`, `Room`, `Booking`, and `Payment` objects.
- **DAO/Repository Layer:** Performs all persistence operations with the OODBMS.

5. OODBMS Communication

Storing a booking:

Reservation UI



ReservationService



Booking Object



BookingRepository



OODBMS

Retrieving available rooms:

OODBMS



RoomRepository



Room Objects



RoomService

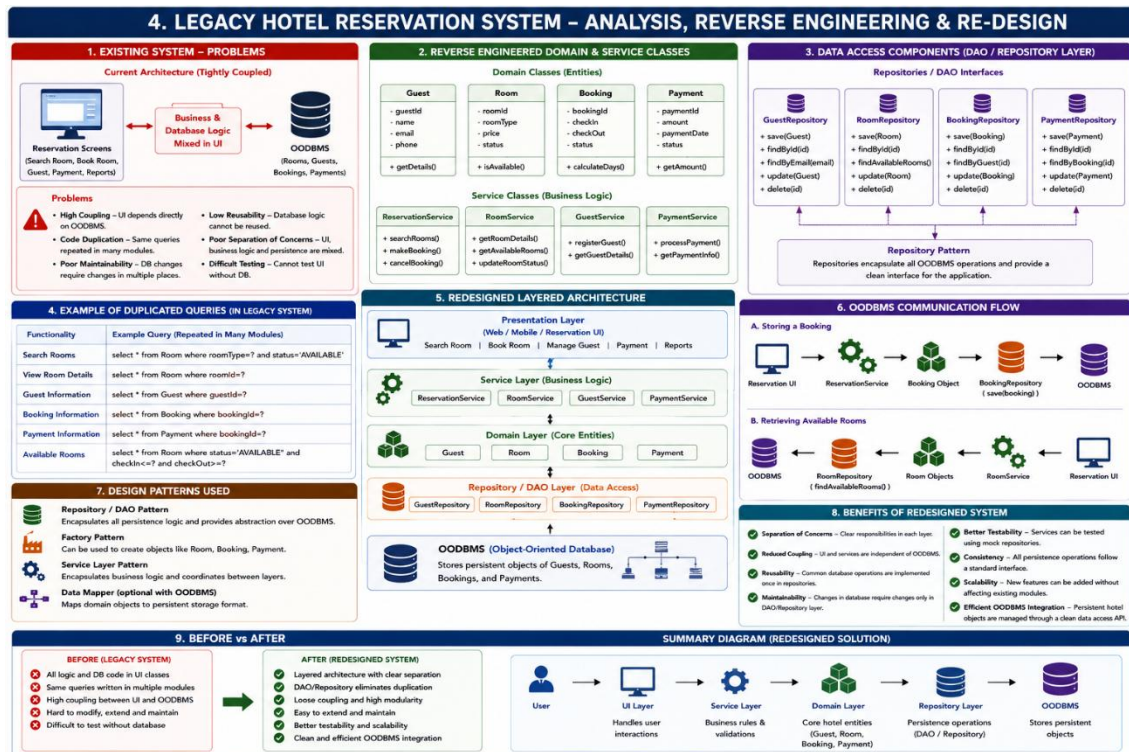


Reservation UI

6. Benefits of the Redesigned System

- **Modularity:** Each layer has a clearly defined responsibility.
- **Reusability:** Repository methods such as `findAvailableRooms()` can be reused by different services.

- **Reduced Duplication:** Common database operations are implemented once.
- **Low Coupling:** UI and business logic are independent of the OODBMS implementation.
- **Maintainability:** Database-related changes are mostly isolated to the DAO/Repository layer.
- **Better Testing:** Services can be tested independently using mock repositories.
- **Efficient OODBMS Integration:** Persistent hotel objects are managed through a consistent data-access interface.



5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

Answer:

1. Problems in the Existing System

- **Duplicated object creation:** Each module contains its own creation logic.
- **High coupling:** Application code directly depends on specific transportation classes.
- **Poor extensibility:** Adding a new service requires changes in multiple modules.
- **Low code reuse:** Common transportation behavior is repeated.
- **Difficult maintenance:** Changes to vehicle creation affect several parts of the system.

2. Reverse-Engineered Classes

Common Abstraction:

Transportation

Attributes:

- vehicleId
- capacity
- fare
- status

Methods:

- book()
- cancel()
- calculateFare()
- getDetails()

Concrete Classes:

- Bus
- Train
- Taxi
- RentalVehicle

Each class implements the common Transportation interface/class.

3. Object Relationships

Transportation

/ | | \

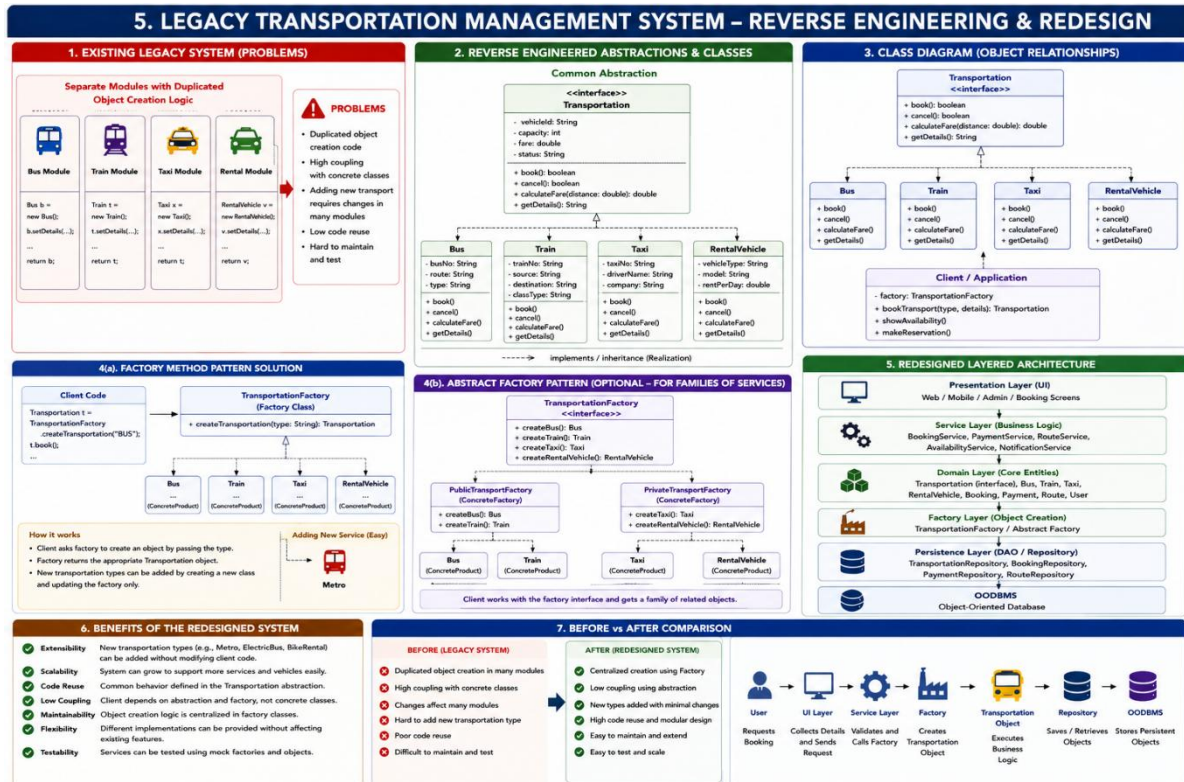
/ | | \

Bus Train Taxi RentalVehicle

4. Benefits of the Redesigned System

- **Extensibility:** New transportation types such as Metro, ElectricBus, or BikeRental can be added easily.

- **Scalability:** The system can support more transportation services without modifying existing client code.
- **Code Reuse:** Common operations are defined once in the Transportation abstraction.
- **Low Coupling:** Client code depends on the factory and abstraction instead of concrete classes.
- **Maintainability:** Object creation is centralized in factory classes.
- **Flexibility:** Different transportation implementations can be selected dynamically.



6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

Answer:

1. SOLID Principle Violations

SRP – Single Responsibility Principle:

A single class may collect patient data, process sensor readings, generate alerts, store records, and produce reports. These are separate responsibilities and should be divided into different classes.

OCP – Open/Closed Principle:

Adding a new monitoring device or sensor requires modifying existing classes. The system should allow new devices to be added without changing existing code.

DIP – Dependency Inversion Principle:

Business classes directly depend on concrete devices and databases. They should instead depend on **interfaces/abstractions**.

2. Reverse-Engineered Classes**Domain Classes:**

- Patient – patientId, name, age, contact
- SensorReading – readingId, type, value, timestamp
- MedicalRecord – recordId, diagnosis, treatment
- Alert – alertId, type, message, severity
- Report – reportId, date, summary

Service Classes:

- PatientService – manages patient information.
- MonitoringService – processes sensor readings.
- AlertService – generates and manages alerts.
- MedicalRecordService – manages medical records.
- ReportService – generates reports.

3. Suitable Design Patterns**Strategy Pattern:**

Can be used for different methods of processing sensor readings or detecting abnormal health conditions.

Factory Pattern:

Can create different monitoring devices without exposing their creation logic.

Observer Pattern:

Can notify doctors, nurses, or caregivers whenever an abnormal sensor reading generates an alert.

4. Redesigned Architecture

Presentation Layer



Service Layer



Domain Layer



Interfaces / Abstractions



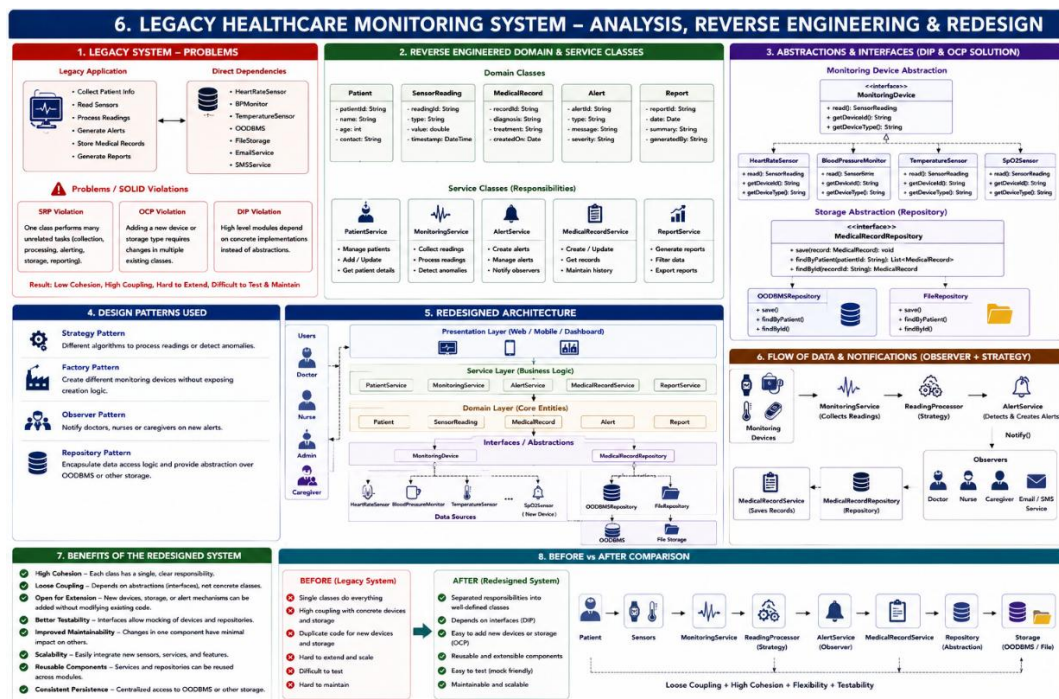
Repository / Device Implementations



OODBMS / Monitoring Devices

5. Improvements After Redesign

- **High Cohesion:** Each class has one clear responsibility.
- **Loose Coupling:** Services depend on interfaces rather than concrete devices or databases.
- **Flexibility:** New sensors and storage technologies can be added without changing business logic.
- **Testability:** Interfaces allow mock sensors and repositories to be used during testing.
- **Maintainability:** Changes to one component have minimal impact on other components.
- **Scalability:** New monitoring devices, alert mechanisms, and healthcare services can be integrated easily.



RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation